规格驱动开发

第04讲 · 2026年9月10日

layout: default

当前位置

今天：提示词杠杆

- L03 给了你三个杠杆和一个指令：要具体
- 仍占主导地位的失败原因是缺少上下文
- 结构良好的提示词也会改变你管理上下文的方式

具体是那个指令。今天它变成了一种纪律。

layout: default

四件事

今天你带走的东西

<v-clicks>

- 精确提示词的词汇
- 将提示词扩展到单个请求之外的结构
- 何时值得编写规格说明，何时不值得
- 架构师模式，一次性运行规格说明

</v-clicks>

这里的所有内容都面向 Lab01 和随后的构建。

layout: section

IDK

信息密集关键词

layout: two-col

模糊

```
Add another way
to output the data.
```

模型猜测每一个细节。

::right::

富含 IDK

```
CREATE output_format.py with
  format_as_html(...) -> str
UPDATE main.py: ADD html
  to --output-format choices
```

没有什么需要猜测。通常一次就对了。

layout: default

动作关键词

词汇表

<ph-plus-circle-bold class="text-2xl text-amber-600 shrink-0" />
CREATE · ADD
CREATE 创建不存在的东西。ADD 向已存在的东西追加内容。
<ph-pencil-simple-bold class="text-2xl text-blue-600 shrink-0" />
UPDATE · REPLACE
UPDATE 原地修改。REPLACE 替换实现。
<ph-minus-circle-bold class="text-2xl text-blue-600 shrink-0" />
DELETE · REMOVE
DELETE 移除整个东西。REMOVE 移除一行或一个参数。

layout: statement

`[动作] [位置]: [细节]`

layout: default

示例

实践中的模式

```
CREATE main.py with
  analyze_transcript(filepath: str) -> TranscriptAnalysis

UPDATE output_format.py: ADD
  format_as_yaml() after format_as_json()

DELETE main.py: REMOVE
  the unused print_debug() function
```

layout: default

IDK 为何有效 — L02 回调

缩小采样分布

- 模糊提示词 → 宽分布 → 模型选择训练先验
- 精确提示词 → 窄分布 → 模型选择你说的内容
- IDK 有效是因为*机制*，而不是英语流利度

layout: section

提示词层级

高 · 中 · 低

layout: default

三个层级

在哪里操作

<ph-cloud-bold class="text-3xl text-blue-600 mb-3" />
高
"让它更好。"自然，但有风险。
<ph-target-bold class="text-3xl text-amber-600 mb-3" />
中 · 甜蜜点
"UPDATE chart.py: word_count_bar_chart() — 上四分位数绿色，降序排列。"
<ph-keyboard-bold class="text-3xl text-blue-600 mb-3" />
低
"第47行，把 X 替换成 Y。"冗长，像打字员。

layout: section

计划 = 提示词

规划瓶颈

layout: default

2024 vs 2026

瓶颈已经转移

<v-clicks>

- **2024**: AI 能理解我想要什么吗？
- **2026**: 我*能*说清楚我想要什么吗？
- 输出质量受限于计划质量
- AI = 执行者。你 = 架构师。
- 人类承包商在规格说明模糊时会提问。LLM 会猜测。

</v-clicks>

layout: statement

规格说明**就是**提示词。

layout: default

决策规则

何时使用规格说明

<ph-check-bold class="text-3xl text-emerald-600 mb-3" />
是
3+ 文件协调 · 相互依赖的步骤 · 可复现性 · 从零开始构建 · 归档
<ph-x-bold class="text-3xl text-rose-600 mb-3" />
否
单个 bug 修复 · 单函数重构 · "既然你在这里，也把 X 做了"
如果你会为人类承包商编写设计文档，就写一个规格说明。

layout: default

你熟悉的代码库

taskr

你一直生活在其中的任务管理器。`task.py` 是记录，`store.py` 是磁盘上的 JSON，`cli.py` 是命令。目前它只能 `add` 和 `list`。

- `taskr done 3` — 标记任务完成
- `taskr rm 3` — 删除任务
- `taskr list --tag work` — 按标签过滤
- `taskr add "..." --due fri` — 截止日期，在列表中显示
- `taskr list --json` — 机器可读的输出

选一个，或者带你自己的。无论哪种方式，命名文件和函数。

layout: statement

聊天休息 · 5 分钟

选一个你想对 `taskr` 做的改动。用 动作 位置: 细节 的格式写出来，然后是*仍然一次成功的最高层级*。

layout: section

PollEv

https://PollEv.com/marcusdarden365

layout: section

规格说明结构

五个部分

layout: default

结构

规格说明文档

<v-clicks>

1. **高层目标** — "是什么"和"为什么"，一般性陈述
2. **中层目标** — 3-5 个可衡量的结果
3. **实现说明** — 依赖项、标准、约束
4. **上下文** — 起始和结束文件
5. **有序的低层任务** — 富含 IDK，按依赖排序

</v-clicks>

layout: default

第1到第3部分

目标和说明

<ph-compass-bold class="text-3xl text-blue-600 mb-3" />
高层
一句话说明原因。"为转录分析 CLI 添加词频可视化和结构化文件输出。"
<ph-check-bold class="text-3xl text-blue-600 mb-3" />
中层
三到五个结果，每个都可以独立验证。图表函数、输出格式化器、两个新的 CLI 标志。
<ph-wrench-bold class="text-3xl text-amber-600 mb-3" />
说明
使用 matplotlib，它已经是依赖项。遵循项目中的模式。简要注释每个新函数。
目标说明完成时是什么样子。说明说明模型不能猜测什么。

layout: default

第4部分 · 最常被跳过的部分

上下文：起始 + 结束

```
### 起始上下文
- src/main.py
- src/data_types.py（只读）
- pyproject.toml（只读）

### 结束上下文
- src/main.py（已更新）
- src/chart.py（新文件）
- src/output_format.py（新文件）
```

消除歧义。模型确切知道什么在范围内。

layout: default

第5部分 · 有序任务

复合序列

```
1. CREATE output_format.py with
   format_as_text/json/md/yaml

2. CREATE chart.py with
   create_bar/pie/line_chart —
   colors: top green, bottom red

3. UPDATE main.py: ADD --chart,
   --output-format, call new fns
```

任务3引用任务1的输出，因为任务1先运行。

layout: section

架构师模式

规格说明在行动中

layout: default

推理 + 编辑

Aider 中的架构师模式

```
aider --architect \
      --model ollama_chat/qwen3.5:9b \
      --editor-model ollama_chat/qwen3.5:9b
```

- **架构师** — 用散文起草计划
- **编辑器** — 将计划转化为编辑
- `auto-accept-architect: false` — 计划停止。阅读它。

两个角色用同一个模型。演示的配置设置了全部三个，所以在屏幕上你会看到只是 `aider src/main.py src/arg_parse.py`。

layout: default

现场演示

看它运行

<v-clicks>

1. `spec.md` 在屏幕上。走过五个部分。
2. `aider src/main.py src/arg_parse.py` — 架构师模式，无标志。
3. 粘贴规格说明。架构师用散文写一个计划。
4. **大声朗读计划，然后按 `y`。**磁盘上还没有任何内容。
5. 四个文件：`output_format.py`， `chart.py` 新文件；`arg_parse.py`， `main.py` 已更新。
6. `uv run main transcript.txt --chart bar --output-format json`

</v-clicks>

layout: section

常见错误

在你写自己的之前

layout: default

常见规格说明错误

评分者看到的情况

<v-clicks>

- **模糊的中层目标** — "让它更好"悄悄溜进来了
- **缺少上下文部分** — 模型触碰了它不应该触碰的文件
- **任务顺序错误** — 任务1引用任务3
- **任务中的代码** — 实现，而不是指令

</v-clicks>

规格说明是沟通工件。这样对待它们。

layout: default

回顾

带走什么

<v-clicks>

- **IDK** 命名了动作和位置，这缩小了采样分布
- **中层** 是甜蜜点，甜蜜点随模型移动
- **计划 = 提示词。** 规格说明是指令，不是关于它的文档
- **有序任务** 复合，所以后面的步骤依赖前面的步骤
- **架构师模式** 将规划与编辑分开。一次一个工作
- **规格说明在三个或更多协调文件上才值得花费成本**

</v-clicks>

layout: default

给房间的

三个问题

1. 规格说明和一个非常长的提示词有什么区别？
2. 你会为一个 bug 修复写规格说明吗？它什么时候变得过度？
3. 为什么把"注释每个函数"放在规格说明里，而不是指望它？

layout: default

本周

Lab01 + 项目构建

<v-clicks>

- **明天（周五 9月11日）** — Aider 设置门控 + 全部十六节课
- **9月14日周一 / 9月15日周二** — Lab01， pivot 实验课
- **10月6日周二** — 项目构建截止，晚上11:59
- **还没有** — `aider-practice` 本周结束；项目构建在 Lab01 之后开始

</v-clicks>

layout: default

项目构建 · 10月6日周二截止

七个特性

一个终端编码助手：对话、上下文中的文件、模型编写的编辑应用在 diff 之后。

	所需行为
F1	对话、预算处理、流式回复
F2	添加、删除、列出和清除上下文
F3	当前文件内容，一致渲染
F4	你测试过的编辑块格式提示词
F5	带有有用错误的解析
F6	精确编辑，完整预览，显式批准

layout: default

两者都评分

这贡献给什么

<ph-package-bold class="text-3xl text-blue-600 mb-3" />
你项目构建的规格说明文档
七个特性且没有 starter 代码是"三个或更多文件，从零开始"的情况。规格说明是必需的交付物。
<ph-code-bold class="text-3xl text-amber-600 mb-3" />
特性 F4，编辑块提示词
为一个小模型写的规格说明。让它模糊，你的解析器就会拒绝输出，所以没有任何东西落在磁盘上。
今天的一切即将变成你提交的代码。

layout: default

下节课

现代智能体 CLI

- 你今天写的同一个规格说明，交给 Claude Code 和 Codex
- 讲台上的演示。不需要账户，没有人被要求购买任何东西
- 更多的自主性，不同的行为，而且三大杠杆仍然适用

你即将构建的东西的成熟版本。周二。

layout: end